

Q.No:5

Question description

Given an array of n integers, your task is to process q queries of the form: what is the minimum value in range $[a, b]$?

Constraints

- $1 \leq n, q \leq 10^5$
- $1 \leq x_i \leq 10^9$
- $1 \leq a \leq b \leq n$

Input

The first input line has two integers n and q : the number of values and queries.

The second line has n integers $x_1, x_2, \dots, x_n$: the array values.

Finally, there are q lines describing the queries. Each line has two integers a and b : what is the minimum value in range $[a, b]$?

Output

Print the result of each query.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
8 4
3 2 4 5 1 1 5 3
2 4
5 6
1 8
3 3
```

EXPECTED OUTPUT

```
2
1
1
4
```

Test Case 2

INPUT (STDIN)

```
10 4
13 12 14 51 11 1 5 3 4 6
3 4
3 6
4 8
1 3
```

EXPECTED OUTPUT

```
14
1
1
12
```

Code:

```
#include <stdio.h>

int tree[400005];
int a[100005];

void build(int *aa, int k, int l, int r)
{
    if (l == r)
    {
        tree[k] = aa[l];
        return;
    }

    int mid = (l + r) / 2;

    build(aa, 2 * k, l, mid);
    build(aa, 2 * k + 1, mid + 1, r);

    if (tree[2 * k] < tree[2 * k + 1])
        tree[k] = tree[2 * k];
    else
        tree[k] = tree[2 * k + 1];
}
```

```
int query(int k, int l, int r, int ql, int qr)
{
    if (qr < l || r < ql)
        return 1000000000;

    if (ql <= l && r <= qr)
        return tree[k];

    int mid = (l + r) / 2;

    int left = query(2 * k, l, mid, ql, qr);
    int right = query(2 * k + 1, mid + 1, r, ql, qr);

    if (left < right)
        return left;
    else
        return right;
}

int main()
{
    int n, q;

    scanf("%d %d", &n, &q);

    for (int i = 1; i <= n; i++)
    {
        scanf("%d", &a[i]);
    }

    build(a, 1, 1, n);

    int ql[100005], qr[100005];

    for (int i = 0; i < q; i++)
    {
        scanf("%d %d", &ql[i], &qr[i]);
    }

    printf("Output:\n");

    for (int i = 0; i < q; i++)
    {
        printf("%d\n", query(1, 1, n, ql[i], qr[i]));
    }

    return 0;
}
```

Output:

Test_case 1:

```
aadityajaiswal@Aadityas-Mac  
8 4  
3 2 4 5 1 1 5 3  
2 4  
5 6  
1 8  
3 3  
Output:  
2  
1  
1  
4
```

Test_case 2:

```
10 4  
13 12 14 51 11 1 5 3 4 6  
3 4  
3 6  
4 8  
1 3  
Output:  
14  
1  
1  
12
```